3 SORTING AND SEARCHING ALGORITHMS

Whenever we want to find information, we carry out a search. Just imagine the number of people around the world who are using the same search engine at the same time. Without efficient searching algorithms, we would have to wait a long time to be shown the results.

We are on all sorts of lists (for example, school and college, clubs and groups, voting registers) and all these must be searched to find relevant information. Sorting information is also important to facilitate efficient searches.

LEARNING OBJECTIVES

- Understand how standard algorithms work (bubble sort, merge sort, linear search, binary search)
- Understand how the choice of algorithm is influenced by the data structures and data values that need to be manipulated
- Evaluate the fitness for purpose of algorithms in meeting specified requirements efficiently, using logical reasoning and test data

Two of the most common tasks in computer programs are sorting data into a particular order and searching for particular items of information.

There might be millions of items of stored data and searching for information wouldn't be efficient if the data was not sorted. Imagine the confusion and difficulty of having to find something in a dictionary that wasn't in alphabetical order. Or planning a trip with train timetables that weren't sorted into time order. Even small lists such as football league tables or the Top 20 music charts are much more useful if they are sorted into order.

SORTING ALGORITHMS

As sorting is such a widely used procedure, many algorithms have been created to carry it out. As with all algorithms, some are more efficient than others.

SUBJECT VOCABULARY

ascending order this is arranging items from smallest to largest (e.g. 1, 2, 3)

descending order this is arranging items from largest to smallest (e.g. 3, 2, 1)

traversal travel across or through something.

BUBBLE SORT

When data is sorted, different items must be compared with each other and moved so that they are in either **ascending order** or **descending order**.

The bubble sort algorithm starts at one end of the list and compares pairs of data items. If they are in the wrong order, they are swapped. The comparison of pairs continues to the end of the list, each complete **traversal** of the list being called a 'pass'. This process is repeated until there have been no swaps during a pass. This indicates that the items must all be in the correct order.

The algorithm can be described as follows.

BUBBLE SORT (ASCENDING ORDER)

1. Start at the beginning of the list.
2. Compare the values in position 1 and position 2 in the list – if they are not in ascending order then swap them.
3. Compare the values in position 2 and position 3 in the list and swap if necessary.
4. Continue to the end of the list.
5. If there have been any swaps, repeat steps 1 to 4.

WORKED EXAMPLE

Here is an example of a bubble sort in action.

Pass 1

4	2	6	1	3	Items 1 and 2 must be swapped.
2	4	6	1	3	Items 1 and 2 are swapped.
2	4	6	1	3	Items 2 and 3 are already in ascending order.
2	4	6	1	3	Items 3 and 4 must be swapped.
2	4	1	6	3	Items 3 and 4 have been swapped.
2	4	1	6	3	Items 4 and 5 must now be swapped.
2	4	1	3	6	Items 4 and 5 have been swapped.

Pass 2

2	4	1	3	6	Items 1 and 2 are in correct order.
2	4	1	3	6	Items 2 and 3 must be swapped.
2	1	4	3	6	Items 2 and 3 have been swapped.
2	1	4	3	6	Items 3 and 4 must be swapped.
2	1	3	4	6	Items 3 and 4 have been swapped.
2	1	3	4	6	Items 4 and 5 do not need to be swapped.

Pass 3

2	1	3	4	6	Items 1 and 2 must be swapped.
1	2	3	4	6	Items 1 and 2 have been swapped.
1	2	3	4	6	All items are now in the correct order.

▲ **Figure 1.8** A bubble sort

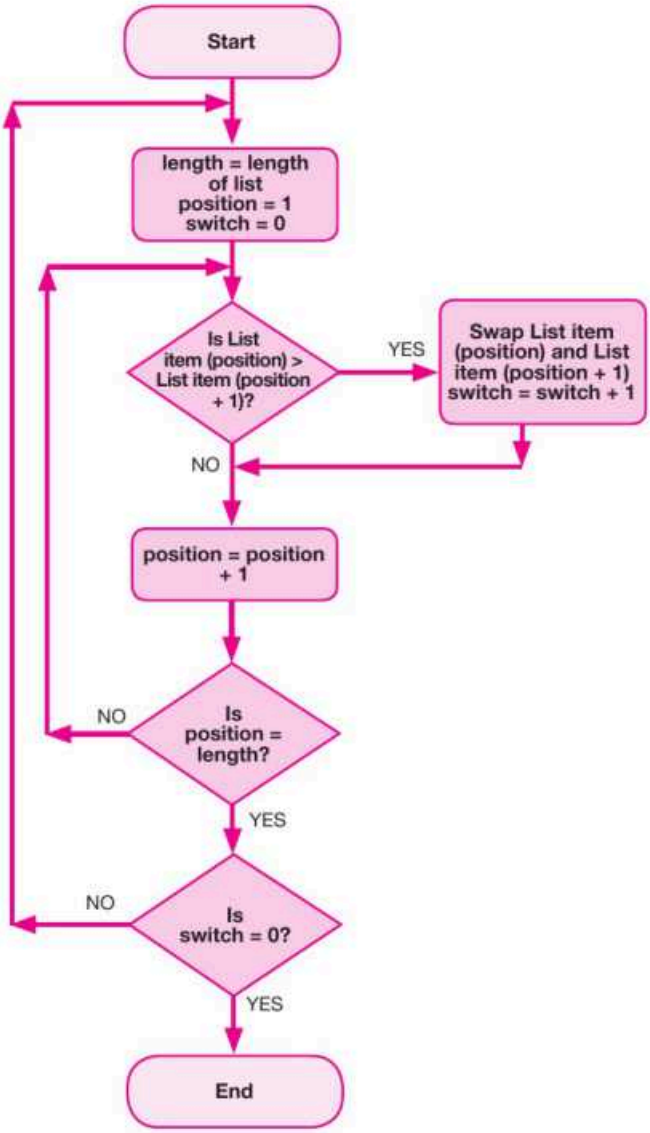
DID YOU KNOW?

Do you know why it is called 'bubble sort'? If you look carefully, you can see that the largest items gradually move to the end, like bubbles rising in water. After the first pass, the largest number is in its correct position. Then after the second pass, the next largest is in its correct position. This happens on each pass and so if the algorithm is to be made more efficient the last set of comparisons can be left out.



It would take a human three passes to carry out this bubble sort. A computer would need four passes because it must continue until there have been no swaps; it cannot just look at all of the numbers at once and see that they are all in order.

The bubble sort algorithm can be represented as a flowchart as shown in Figure 1.9.



▶ **Figure 1.9** Bubble sort algorithm written as a flowchart

SKILLS CRITICAL THINKING

ACTIVITY 9

HOW DOES BUBBLE SORT WORK?

Study the flowchart of the bubble sort algorithm.

Using the variables declared, can you explain the logic behind the algorithm? How does it function to sort a list?

GENERAL VOCABULARY

repeatedly many times

SUBJECT VOCABULARY

recursion a process that is repeated. For example, a document can be checked and edited, checked and edited and so on until it is perfect

MERGE SORT

Merge sort is a sorting algorithm that divides a list into two smaller lists and then divides these until the size of each list is one. **Repeatedly** applying a method to the results of a previous application of the method is called **recursion**.

In computing, a problem is solved by repeatedly solving smaller parts of the problem. A part of a program can be run and rerun on the results of the previous run (e.g. repeatedly dividing a number by 2).

WORKED EXAMPLE

Here is an example of a merge sort which will sort the following list into ascending order.



The list is split into half with recursion to produce a left list and a right list each time.

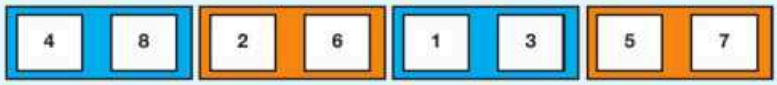




This continues until there is only one item in each list. Therefore, each list is sorted into order.



The left and right lists are now merged through recursion with the items in the correct order.



The leftmost items in each list are the lowest items of those lists and the algorithm compares them – in this case 4 with 2. The 2 is inserted in the new list and the 4 is then compared with the second number of the right list – 6. The 4 is inserted and the 6 is compared with the second number of the left list.



The algorithm now merges these two lists in the same way to produce the final sorted list. 1 is compared with 2 and then 2 with 3, 3 with 4, etc.



KEY POINT

In the exam, you will be expected to show the intermediate stages when the algorithms are applied to data.

SKILLS PROBLEM SOLVING

ACTIVITY 10

USING MERGE SORT

Using a table like the one in the worked example on page 18, show how the following list would be sorted into descending order using merge sort.

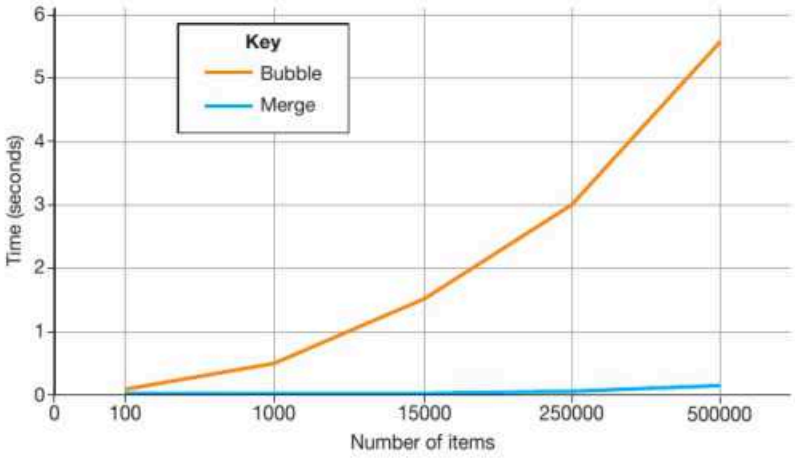
48, 20, 9, 17, 13, 21, 28, 60

EXTEND YOUR KNOWLEDGE

Only two sorting algorithms are required for the specification: bubble sort (the slowest) and merge sort (one of the most efficient). There are far more, and many of them are relatively easy to code. Research the insertion and selection sorts.

EFFICIENCY OF SORTING ALGORITHMS

This graph compares the performance of the bubble and merge sort algorithms.



▶ **Figure 1.10** A graph comparing the performance of bubble and merge sort algorithms

SUBJECT VOCABULARY

brute force an algorithm design that does not include any techniques to improve performance, but instead relies on computing power to try all possibilities until the solution to a problem is found

divide and conquer an algorithm design that works by dividing a problem into smaller and smaller sub-problems, until they are easy to solve. The solutions to these are then combined to give a solution to the complete problem

The bubble and merge sort algorithms demonstrate two alternative approaches to algorithm design.

The bubble sort algorithm is said to be using **brute force** because it starts at the beginning and completes the same task over and over again until it has found a solution.

The merge sort uses the **divide and conquer** method because it repeatedly breaks down the problem into smaller sub-problems, solves those and then combines the solutions.

The graph shows that a bubble sort is far slower at sorting lists of more than 1000 items, but for smaller lists the time difference is too small to be of importance.

As the bubble sort algorithm is easier to code, it could be beneficial to use it for smaller lists of less than 1000 items.

SEARCHING ALGORITHMS

To find a specific item in a list involves carrying out a search. Like sorting, some methods of searching are more efficient than others.

LINEAR SEARCH

A linear search is a simple algorithm and not very sophisticated. It simply starts at the beginning of the list and goes through it, item by item, until it finds the item it is looking for or reaches the end of the list without finding it.

GENERAL VOCABULARY

sequential following in order, one after the other

A linear search is **sequential** because it moves through the list item by item.

LINEAR SEARCH

1. Start at the first item in the list.
2. Compare the item with the search item.
3. If they are the same, then stop.
4. If they are not, then move to the next item.
5. Repeat 2 to 4 until the end of the list is reached.

BINARY SEARCH

Like a merge sort, a binary search uses a 'divide and conquer' method.

In a binary search the middle or **median** item in a list is repeatedly selected to reduce the size of the list to be searched – another example of recursion. If the selected item is too high or too low, then the items below or above that selected item can be searched.

To use this method, the list must be sorted into ascending or descending order. It will not work on an unsorted list.

SUBJECT VOCABULARY

median the middle number when the numbers are put in ascending or descending order (e.g. if there are 13 numbers, then the 7th number is the median). If there are an even number of items in a list, the median is the mean of the middle two numbers (e.g. if there are 10 numbers, add the 5th and 6th numbers together and divide the result by 2). In a binary search, the higher of the two numbers would be chosen

DID YOU KNOW?

You have probably used a binary search method when trying to guess a number between two limits. If you are asked to guess a number between 1 and 20 you will probably start at 10, the middle number. If you are told this is too high, you will then guess 5, the middle number between 1 and 10, and then repeat this method until you find the correct one.

BINARY SEARCH (ITEMS IN ASCENDING ORDER)

1. Select the median item of the list.
2. If the median item is equal to the search item, then stop.
3. If the median is too high, then repeat 1 and 2 with the sub-list to the left.
4. If the median is too low, then repeat 1 and 2 with the sub-list to the right.
5. Repeat steps 3 and 4 until the item has been found or all of the items have been checked.

WORKED EXAMPLE

In this list, the search item is the number 13.

3	13	24	27	(31)	39	45	60	69	SELECT THE MEDIAN NUMBER.

As this is too high, the sub-list to the left of the median must be searched.

3	13	(24)	27	THE MEDIAN NUMBER OF THIS SUB-LIST IS NOW SELECTED.

This is again too high and so the sub-list to the left must be searched.

3	(13)	THE MEDIAN NUMBER IS NOW THE SEARCH ITEM.

▲ **Figure 1.11** Binary search including sub-lists

In this example, it took three attempts to find the search item. A linear search would have accomplished this with only two attempts.

SKILLS PROBLEM SOLVING

ACTIVITY 11

USING BINARY SEARCH

Display the stages of a binary search, as in the worked example above, to find the number 13 in this list.

3 9 13 15 21 24 27 30 36 39 42 54 69

Compare your results with those of others in your group. Are all your answers the same?

EFFICIENCY OF SEARCHING ALGORITHMS

In the example on page 20, the linear search was more efficient because it only had to carry out two comparisons instead of the three for a binary search. But is this always the case?

Searching algorithms can be compared by looking at the 'worst case' and the 'best case' for each one.

WORKED EXAMPLE

If you wanted to find a particular item in a list of 1000 items, these are the best- and worst-case scenarios for the linear search and binary search algorithms.

Linear search

A linear search starts at the first item and then works through sequentially.

The best case would be if the item is first in the list.

The worst case would be if it is last in the list.

Therefore, in this example the average would be 500 comparisons.

Binary search

The best case would be if the item is in the median position in the list. The search would require only one comparison.

For the worst case it would have to choose the following medians until it finally hit the target.

(This assumes that the target is always smaller than the median.)

Attempt	Median
1	500
2	250
3	125
4	63
5	32
6	16
7	8
8	4
9	2
10	1

Therefore, the worst case for the binary search is ten comparisons.

The binary search is therefore far more efficient than the linear search.

So, should a binary search be used every time? That depends on the circumstances. The binary search has one great disadvantage. The list must be already sorted into ascending or descending order. Therefore, a sorting algorithm must be applied before the search.

If the list is to be searched just once then a linear search would be better, but if there is a large list that will be searched many times then sorting the list and using a binary search would be better. Once the list has been sorted, new items can be inserted into the correct places.

CHECKPOINT

Strengthen

SKILLS CRITICAL THINKING

S1 What are the differences between the 'bubble sort' and 'merge sort' algorithms?

SKILLS DECISION MAKING

S2 How does a binary search algorithm find the search item?

Challenge

SKILLS CRITICAL THINKING

C1 When might a linear search be preferable to a binary search, even if the binary search algorithm is more efficient?

How confident do you feel about your answers to these questions? If you're not sure you answered them well, try the following activities again.

- For S1 have a look at the 'Sorting algorithms' section.
- For S2 have a look at the 'Binary search' section.

SUMMARY

- There are many algorithms for sorting and searching data.
- The choice of algorithm depends on the data that is to be processed.
- If only a small amount of data needs to be processed, then a simpler, but less efficient search algorithm may be the best choice. The time difference of the search or sort time will be negligible.